A SCALABLE FRAMEWORK FOR STRUCTURED DATA CLEANING IN STREAMING ENVIRONMENTS: AN ML-BASED PREDICTION APPROACH WITH HUMAN-IN-THE-LOOP GOVERNANCE

Arachchi L.T.E.

(IT22893666)

BSc (Hons) degree in Information Technology Specializing in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

A SCALABLE FRAMEWORK FOR STRUCTURED DATA CLEANING IN STREAMING ENVIRONMENTS: AN ML-BASED PREDICTION APPROACH WITH HUMAN-IN-THE-LOOP GOVERNANCE

Arachchi L.T.E.

(IT22893666)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology Specialized in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

DECLARATION

I declare that this dissertation is my own work and does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

NAME	STUDENT ID	SIGNATURE	DATE
Arachchi L.T.E.	IT22549136		22 / 04 / 2026

The above candidate has carried out research for the Bachelor's degree dissertation under my supervision.

Name of the Supervisor: Dr. Lakmini Abeywardhana

Signature of Supervisor: _____

Date: 22 / 04 / 2026

Name of the Co-Supervisor: Dr. Mahima Weerasinghe

Signature of Co-Supervisor: _____

Date: 22 / 04 / 2026

DEDICATION

This dissertation is dedicated to the data engineering and machine learning research communities whose collective efforts to advance intelligent, accountable, and explainable data management have made work such as this possible. It is also dedicated to domain experts and practitioners who bear the daily responsibility of maintaining data quality in production systems — your operational challenges are the motivation behind every design decision in this framework.

To every researcher working to bridge the gap between fully automated AI systems and meaningful human oversight — this work is a contribution to the conviction that automation and accountability are not in conflict, but are mutually reinforcing.

ACKNOWLEDGEMENT

The authors acknowledge the open-source communities behind Apache Kafka, FastAPI, scikit-learn, SHAP, LIME, and the Groq API, whose infrastructure underpins the proposed framework. Feedback from data engineering practitioners on the HITL governance workflow substantially informed the design of the review interface and the inactivity fallback mechanism.

Sincere gratitude is expressed to the research communities behind HoloClean, ActiveClean, and Table-GPT, whose foundational contributions defined the state of the art that this framework builds upon and extends. The detailed evaluation methodology was shaped significantly by the rigorous experimental designs published in the data cleaning and streaming data quality literature.

The authors also acknowledge the contributions of the Groq platform team, whose low-latency LLaMA 3.3-70B inference API made the real-time rule extraction component of this framework operationally feasible within the latency constraints of a streaming pipeline. Without sub-second LLM inference, the rule extraction module would have been incompatible with the batch processing architecture.

Appreciation is extended to the supervisory team and the panel of academic reviewers whose critical feedback across multiple stages of this research sharpened the technical arguments, improved the experimental design, and significantly raised the quality of the final manuscript. Their expertise in machine learning, data systems, and human-computer interaction provided invaluable interdisciplinary perspective.

Finally, sincere thanks are extended to all individuals — colleagues, family members, and academic peers — who provided encouragement, constructive criticism, and practical assistance throughout the preparation of this dissertation. Your support has been deeply appreciated.

ABSTRACT

Maintaining high data quality within real-time streaming pipelines presents an ongoing and multidimensional challenge. The continuous, high-volume, and heterogeneous character of streaming data means that quality degradation — in the form of missing values, statistical outliers, categorical violations, and schema inconsistencies — is not a recoverable exception but a routine operational condition. Unlike batch-oriented settings where remediation can be deferred to a scheduled correction pass, streaming environments demand immediate, in-flight corrections that prevent cascading quality degradation across downstream consumers including machine learning models, reporting dashboards, and decision support systems.

This dissertation presents a novel scalable framework for structured data cleaning in streaming environments that moves beyond conventional rule-based and statistical methods by employing dependency-aware machine learning prediction as its primary correction mechanism. The central innovation is a per-column Random Forest prediction engine in which each target attribute is corrected by a dedicated model trained exclusively on statistically correlated columns, identified through three complementary dependency measures: chi-square independence testing for categorical-categorical column pairs, normalized mutual information scoring for mixed categorical-numerical pairs, and Spearman rank correlation for ordinal and continuous numerical pairs. This dependency-aware architecture avoids the spurious cross-attribute interference that degrades holistic ML-based repair systems on wide datasets, and yields contextually consistent corrections grounded in genuine inter-column relationships rather than statistical coincidences in the training data.

The framework incorporates four additional layers that work in concert with the ML prediction engine. First, LLM-driven rule extraction via LLaMA 3.3-70B, accessed through the Groq API, automatically derives validation constraints from unstructured domain documents without manual specification. Second, a multi-dimensional anomaly detection component identifies four categories of issue — missing values, statistical outliers, categorical violations, and range constraint breaches — with per-issue explainability payloads generated through SHAP (global feature attribution) and LIME (local linear approximation). Third, a Human-in-the-Loop (HITL) governance layer positions human review as the primary correction pathway through three operational modes: active review, partial review with auto-fill, and inactivity-triggered preview fallback. Fourth, a composite quality scoring mechanism weights Completeness at 40 percent, Validity at 40 percent, and Consistency at 20 percent, normalizing the result to a 0–95 scale with a five-tier grade mapping calibrated to the irreducible noise inherent in high-velocity streaming data.

All components are integrated within a production-grade Apache Kafka streaming backbone implemented using the Confluent Platform, with configurable batch sizes and fault tolerance through atomic file writes and persistent batch storage. The React and Vite frontend hosts the HITL review interface with all-channel inactivity detection. The FastAPI backend manages session state, pipeline orchestration, and both destructive commit and non-destructive preview correction endpoints.

The evaluation framework assesses the proposed approach across six dimensions: rule extraction fidelity, issue detection performance, correction accuracy, dependency graph precision, HITL review efficiency, and system throughput and latency. Based on the theoretical properties of dependency-informed feature selection, the framework is expected to outperform global Random Forest baselines in correction accuracy for datasets exhibiting moderate to high inter-column correlation, while the HITL governance layer is expected to demonstrate measurable quality improvement over fully automated correction for outlier and categorical violation cases where domain knowledge is essential.

Keywords: structured data cleaning, streaming data, machine learning prediction, Random Forest, dependency analysis, Human-in-the-Loop, data quality, LLM rule extraction, SHAP, LIME, Apache Kafka

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CCPA	California Consumer Privacy Act
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
ETL	Extract, Transform, Load
F1	F1-Score (harmonic mean of precision and recall)
FastAPI	Fast Asynchronous Python API Framework
GDPR	General Data Protection Regulation (EU, 2018)
GAIN	Generative Adversarial Imputation Network
HIPAA	Health Insurance Portability and Accountability Act
HITL	Human-in-the-Loop
IQR	Interquartile Range
JSON	JavaScript Object Notation
LLM	Large Language Model
LIME	Local Interpretable Model-Agnostic Explanations
MAE	Mean Absolute Error
MI	Mutual Information
MOA	Massive Online Analysis
ML	Machine Learning
NaN	Not a Number (missing numeric value)
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RF	Random Forest
SHAP	SHapley Additive exPlanations
UUID	Universally Unique Identifier
VLDB	Very Large Data Bases (conference)
XAI	Explainable Artificial Intelligence

01. INTRODUCTION

1.1 Background

The widespread adoption of real-time data streaming across healthcare, finance, the Internet of Things, and e-commerce has fundamentally reshaped data engineering practice. Modern organisations ingest data continuously from dozens of heterogeneous sources — electronic health records, financial transaction processors, IoT sensor networks, customer interaction platforms — each with its own schema conventions, error profiles, and data quality characteristics. The result is a continuous, high-velocity inflow that is inherently vulnerable to quality degradation at every stage of the pipeline.

Three categories of quality defect arise with particular frequency in streaming environments. Missing values emerge from sensor dropouts, network transmission failures, and upstream system outages that produce incomplete records without triggering pipeline alerts. Statistical outliers arise from measurement errors, calibration drift, and genuine distribution shifts in the underlying phenomenon being measured, creating values that are syntactically valid but semantically anomalous. Categorical violations occur when upstream systems evolve their vocabularies — adding new enumeration values, retiring old ones, or changing encoding conventions — without coordinated schema updates propagating to downstream consumers.

Unlike batch-oriented settings where remediation can be deferred to a scheduled correction pass, streaming environments demand immediate, in-flight corrections. A single corrupted record in a financial transaction stream may trigger a cascade of erroneous downstream computations across risk models, fraud detection systems, and regulatory reporting pipelines before a batch correction could be applied. The latency window between defect occurrence and correction must be bounded in milliseconds or seconds, not hours or days.

Existing automated data cleaning methods fall into three broad classes, each carrying significant limitations for the streaming context. Rule-based systems provide formal consistency guarantees but demand extensive manual constraint specification and cannot readily adapt to evolving domains without expert intervention. Statistical imputation techniques achieve strong correction accuracy in batch settings but are computationally impractical for streaming contexts subject to tight per-batch latency constraints. Holistic ML-based repair frameworks such as HoloClean combine probabilistic inference with constraint satisfaction but operate exclusively in batch mode and presuppose manually specified integrity constraints that may not exist for novel or rapidly evolving data sources.

A fundamental weakness shared by all existing ML-based cleaning systems is the failure to account for inter-attribute dependencies when constructing correction models. A model trained to correct one column

using all other columns as predictors will inadvertently exploit spurious correlations — statistical coincidences that exist in the training sample but do not reflect genuine causal or associative relationships. For wide datasets where the number of columns approaches or exceeds the number of clean training rows, this problem is acute: the model faces a high-dimensional input space filled with noise, producing corrections that are statistically coherent in the training distribution but semantically incorrect in the application domain.

A second critical gap is the absence of human governance in existing automated correction systems. Automated corrections, regardless of their statistical accuracy, carry an implicit claim of correctness that is difficult to audit, contest, or trace after the fact. In regulated domains — healthcare, finance, telecommunications — corrections to stored records may have legal consequences, and the ability to demonstrate that a human reviewer approved each correction is a compliance requirement, not a convenience feature. No existing streaming data cleaning framework provides this accountability.

1.2 Problem Statement

The fundamental problem addressed in this dissertation can be stated precisely. Given a continuous stream of structured records arriving in configurable batches, where records exhibit heterogeneous quality defects including missing values, statistical outliers, categorical violations, and range constraint breaches, the goal is to produce a corrected output stream in which each defect has been addressed by the most accurate, contextually appropriate, and accountable correction available.

This problem has five distinct subproblems. The rule specification problem asks how validation constraints should be derived for a new data domain without manual expert specification — an impractical requirement at streaming velocity. The detection problem asks how all four categories of defect should be identified reliably and efficiently within the latency budget of the streaming pipeline. The dependency identification problem asks which columns provide genuine predictive information for correcting each target column, as opposed to spurious correlations that degrade correction quality. The correction problem asks how the identified dependencies should be exploited to produce accurate, contextually consistent corrections. The governance problem asks how human oversight should be integrated into the correction workflow in a way that is operationally feasible at streaming velocity while maintaining per-correction accountability.

This framework addresses all five subproblems through a unified architecture that combines LLM-based rule extraction, statistical dependency graph construction, per-column Random Forest prediction, SHAP and LIME explainability, and a three-mode HITL governance layer within a Kafka-native streaming pipeline.

1.3 Research Scope

The scope of this dissertation is the design, implementation, and evaluation framework design of a scalable structured data cleaning framework covering the following components:

- An LLM-driven rule extraction module using LLaMA 3.3-70B via the Groq API, capable of deriving field-level validation constraints from unstructured domain documents in CSV format.
- A multi-dimensional issue detection module identifying missing values, statistical outliers, categorical violations, and range constraint breaches.
- A statistical dependency analysis module constructing a directed dependency graph using chi-square testing, normalized mutual information, and Spearman rank correlation.
- A per-column ML prediction engine using dependency-validated Random Forest classifiers and regressors as the primary correction mechanism.
- A SHAP and LIME explainability layer generating per-fix attribution payloads for human reviewer consumption.
- A three-mode HITL governance layer with active review, partial review with auto-fill, and inactivity-triggered preview fallback.
- A composite data quality scoring mechanism with Completeness, Validity, and Consistency dimensions normalized to a 0–95 scale.
- A production-grade Apache Kafka streaming backbone with fault tolerance, configurable batch processing, and a React/Vite HITL review frontend.

The scope explicitly excludes the design or fine-tuning of the LLM; the LLaMA 3.3-70B model is accessed as a pre-trained service through the Groq API. The Random Forest implementations use standard scikit-learn defaults with modest hyperparameter adjustments optimized for streaming latency.

1.4 Related Work

1.4.1 Rule-Based Data Cleaning

Systems grounded in integrity constraints and denial constraints provide formal consistency guarantees but place the burden of constraint specification and maintenance entirely on domain experts. Fan et al. demonstrated through both theoretical analysis and empirical evaluation that even carefully designed constraint sets develop coverage gaps as data sources evolve, producing systematic blind spots for quality issues that fall outside the specified constraint vocabulary. The maintenance burden compounds over time as upstream data sources add new fields, modify value ranges, and evolve their categorical vocabularies without triggering constraint updates.

Conditional functional dependencies (CFDs) extended classical functional dependencies by allowing constraints to be conditional on the values of other attributes, improving coverage for semantically rich domains. However, the specification of CFDs requires domain experts with both statistical knowledge and data engineering expertise, and the number of constraints required for comprehensive coverage grows rapidly with dataset width. The present framework addresses this limitation through LLM-based constraint

extraction, which derives validation rules from available documentation across any target domain without manual intervention.

1.4.2 Statistical and ML-Based Repair

Matrix factorization-based imputation methods, which decompose the incomplete data matrix into low-rank factors and use the reconstruction to impute missing values, achieve strong correction accuracy for missing value imputation in batch settings where the entire matrix is available simultaneously. GAIN (Generative Adversarial Imputation Network) extended this to a generative modelling approach, using a generator-discriminator architecture to learn the conditional distribution of missing values given observed values and produce statistically plausible imputations.

HoloClean advanced the state of the art in holistic data repair by coupling extracted integrity constraints with a probabilistic graphical model that enables joint repair of all defects simultaneously. By treating the clean dataset as a hidden variable and observed defects as noisy observations, HoloClean frames data repair as a probabilistic inference problem and applies loopy belief propagation to find the maximum a posteriori repair. However, all three approaches share a fundamental constraint for streaming deployment: computational overhead that scales poorly with dataset dimensionality and batch arrival rate. The proposed framework achieves streaming-compatible latency by training lightweight per-column Random Forest models that are computationally independent and can execute in milliseconds per fix.

1.4.3 Dependency-Aware Correction

The relevance of inter-column dependency structure for correction quality is established in the data cleaning literature. HoloClean encodes dependencies implicitly through integrity constraints — a column is predicted from its constraint neighbours — but this encoding is contingent on the quality and completeness of the manually specified constraints. ActiveClean directs human labelling effort toward the most critical attributes based on their estimated downstream impact on model performance, effectively implementing a form of dependency-aware prioritization without constructing an explicit dependency graph.

The present work makes dependency structure explicit and learnable through a statistical construction process that operates independently of any constraint specification. The dependency graph is built from the data itself using three complementary statistical measures selected to cover the full range of column type combinations: chi-square for categorical pairs, mutual information for mixed pairs, and Spearman correlation for numerical pairs. Each model is then constrained to access only its statistically validated predictors, preventing the spurious corrections that arise when unrelated columns are included in the feature set.

1.4.4 LLMs in Data Preparation

Table-GPT demonstrated that domain-adapted language models fine-tuned on table-understanding tasks could perform a diverse range of table operations including data type inference, missing value imputation, column description generation, and entity disambiguation. The key insight of Table-GPT was that language model pre-training on large text corpora provides substantial implicit knowledge about the semantics of common data fields — names, addresses, dates, monetary amounts — that can be exploited for data preparation tasks without requiring domain-specific training data.

Entity matching through LLMs achieves state-of-the-art performance on standard benchmarks by leveraging semantic understanding to resolve references to the same real-world entity across heterogeneous records. This work extends LLM application in the data preparation domain to validation rule extraction, using the model's semantic understanding to identify and formalize constraints from unstructured domain documentation. Crucially, the extracted rules are integrated into an operational streaming cleaning pipeline — a combination not previously demonstrated in the literature.

1.4.5 Human-in-the-Loop Data Management

HITL methodologies have been applied across data integration, entity resolution, schema governance, and data annotation tasks. The common motivation is that automated systems, regardless of their statistical sophistication, encounter cases where domain knowledge provides information unavailable to any statistical model — cases where the correct answer requires semantic understanding of the business context rather than pattern matching in historical data.

Bontempelli et al. established through empirical study that human oversight is necessary when automated systems encounter knowledge drift — situations where the distribution of the data shifts in ways that render historical training data unrepresentative of the current environment. ActiveClean incorporated human feedback into the model retraining cycle, using reviewer-approved corrections as additional labeled training examples. However, it did not provide decision-level accountability: there was no per-correction record distinguishing which corrections were reviewer-approved, which were model-generated, and which were applied without review. The present framework applies HITL at the level of individual correction decisions and maintains an audit trail that records the governance mode, reviewer identity, and timestamp for every correction applied.

1.4.6 Streaming Data Quality

Real-time data quality monitoring frameworks address anomaly detection in streaming contexts — identifying when the quality of incoming data has degraded below acceptable thresholds — but do not extend to automated correction. These systems can alert operators to quality problems but cannot resolve them. MOA provides an extensible online learning infrastructure for concept drift detection and adaptive classification in streaming settings, but requires manual configuration of detection logic for each monitored attribute.

The proposed framework pushes streaming quality management beyond monitoring into the correction domain, integrating detection, ML-based repair, and HITL governance within a unified Kafka-native pipeline. The key challenge this creates is latency: correction must be completed within the inter-batch interval to prevent backpressure accumulation in the Kafka consumer group. The per-column Random Forest architecture is specifically designed to meet this constraint by training models that are computationally lightweight and independently parallelizable.

1.5 Research Gap

The literature survey reveals a fundamental gap: no existing framework concurrently delivers all five of the capabilities required for production-grade streaming data cleaning with human accountability.

Gap 1 — Automated Rule Derivation: Existing automated correction systems either require manually specified integrity constraints or operate without any rule vocabulary. LLM-based rule extraction has been applied to table understanding tasks but not to operational streaming cleaning pipelines.

Gap 2 — Dependency-Informed ML Correction: Existing ML-based repair frameworks either use all available columns as predictors — risking spurious corrections — or use integrity constraints to implicitly define the predictor set — requiring manual constraint specification. No published framework constructs an explicit statistical dependency graph prior to model training and uses it to constrain the predictor sets of per-column models.

Gap 3 — Per-Correction XAI for Human Review: Existing HITL data cleaning systems present correction suggestions without quantified feature attributions, forcing reviewers to accept or reject suggestions without understanding their reasoning. SHAP and LIME have been applied to ML model explanation broadly but have not been integrated into a streaming data cleaning HITL interface.

Gap 4 — Three-Mode HITL Governance with Inactivity Fallback: Existing HITL systems operate in a binary mode — either the human reviews or automation applies corrections. No published framework

implements a three-mode governance structure that gracefully degrades from full human review to partial auto-fill to inactivity-triggered preview while maintaining per-correction audit accountability.

Gap 5 — Unified Streaming Architecture: Detection, correction, explainability, and governance have been addressed separately in the literature. No published system integrates all four within a single Kafka-native streaming pipeline with a React-based HITL review frontend.

1.6 Research Questions

RQ1: Does dependency-informed feature selection for per-column Random Forest models produce higher correction accuracy than global feature inclusion for datasets exhibiting moderate to high inter-column correlation, and how does this advantage scale with dataset width and the density of the dependency graph?

RQ2: How effectively can LLaMA 3.3-70B via the Groq API extract field-level validation constraints from unstructured domain documents, and what is the precision and recall of the extracted constraints relative to ground-truth expert annotations across healthcare, financial, and IoT domains?

RQ3: Does the combination of SHAP and LIME explanations in the HITL review interface produce measurably faster and more accurate reviewer decisions compared to presenting correction suggestions without explanations?

RQ4: How does the three-mode HITL governance structure affect the distribution of correction decisions across active review, auto-fill, and inactivity-triggered fallback modes, and what is the quality delta between human-selected and auto-selected corrections for each anomaly category?

RQ5: Can the framework sustain the target throughput and latency profiles — sub-second per-batch latency for 500-record batches, linear throughput scaling with Kafka consumer instances — across datasets of varying width (10, 50, 100 columns) and defect density?

1.7 Objectives

1.7.1 Aim

To design, implement, and evaluate a scalable framework for structured data cleaning in streaming environments that combines dependency-aware ML prediction, automated LLM rule extraction, per-correction XAI explanation, and three-mode HITL governance within a production-grade Kafka streaming architecture.

1.7.2 Main Objective

To develop and deploy a structured data cleaning pipeline in which each detected quality defect in an incoming streaming record is addressed by a correction produced from dependency-validated ML prediction, explained to a human reviewer through SHAP and LIME attributions, and committed through an accountable governance pathway that distinguishes human-approved, auto-filled, and fallback-triggered corrections.

1.7.3 Sub-Objectives

- To survey existing rule-based, statistical, ML-based, and LLM-based data cleaning approaches and identify the specific limitations that motivate the dependency-informed architecture proposed in this work.
- To design and implement an LLM-driven rule extraction module using LLaMA 3.3-70B that automatically derives field-level validation constraints from unstructured PDF and plain-text domain documents and serializes them in machine-readable CSV format.
- To design and implement a statistical dependency analysis module that constructs a directed dependency graph using chi-square testing, normalized mutual information, and Spearman rank correlation, with configurable significance thresholds.
- To design and implement a per-column Random Forest prediction engine that trains dedicated classifiers and regressors using dependency-validated predictor sets, with a scikit-learn Pipeline preprocessing architecture handling numerical and categorical features.
- To design and implement a SHAP and LIME explainability module that generates per-fix attribution payloads for both global feature importance and local linear approximation, and renders these attributions in the HITL review interface.
- To design and implement a three-mode HITL governance layer with active review, partial review with auto-fill, and inactivity-triggered preview fallback, including an all-channel inactivity detector and a non-destructive preview endpoint.
- To design and implement a composite data quality scoring mechanism that computes weighted Completeness, Validity, and Consistency sub-scores at two pipeline checkpoints and maps the normalized result to a five-tier grade scale.
- To design an experimental evaluation framework assessing the complete system across six dimensions: rule extraction fidelity, issue detection performance, correction accuracy, dependency graph precision, HITL review efficiency, and system throughput and latency.

02. METHODOLOGY

2.1 System Architecture

The proposed framework adopts a layered microservices architecture that decomposes the data cleaning pipeline into independently scalable components. The architecture spans five layers: a user interaction layer hosting the HITL review interface, an API orchestration layer managing session state and pipeline coordination, a Kafka streaming layer providing fault-tolerant batch transport, an ML and LLM processing layer executing the core cleaning logic, and a persistent storage layer capturing results, rules, and audit trails.

This layered decomposition was chosen deliberately over a monolithic architecture for three reasons. First, each layer scales independently: if rule extraction becomes a throughput bottleneck, additional LLM API workers can be added without modifying the Kafka consumer or the ML engine. Second, each layer can be tested in isolation with well-defined interfaces, improving testability and reducing integration risk. Third, the separation of the HITL review interface from the processing backend ensures that reviewer interactions do not block the pipeline: the React frontend communicates with FastAPI over HTTP, while Kafka handles the internal message passing between pipeline stages asynchronously.

Figure 1-0-aSystem architecture: five layers spanning user interaction, API orchestration, Kafka streaming, ML/LLM processing, and persistent storage.

Table 1Summarizes the component technologies and their responsibilities within the architecture.

Component	Technology	Responsibility
Frontend	React + Vite	HITL review UI; inactivity timer; approve/rollback modal
Backend API	FastAPI (Python)	Session management; pipeline orchestration; fix preview and commit
Streaming	Kafka + Zookeeper	Fault-tolerant batch ingestion; partition-level scalability
ML Engine	Scikit-learn RF	Per-column dependency-trained classifiers and regressors
XAI Layer	SHAP + LIME	Per-fix attributions surfaced in the HITL review interface
LLM Module	Groq / LLaMA 3.3-70B	Automated validation rule extraction from domain documents
Storage	Session store + CSV	In-memory state; audit trail; cleaned dataset export

2.2 Technology Stack

The backend is implemented in Python 3.11 using FastAPI as the asynchronous web framework. FastAPI's native asyncio integration enables concurrent session management across multiple simultaneous cleaning jobs without blocking the API layer while Kafka consumers process batches. The two correction endpoints — POST /fixes/{session_id} for committing corrections and POST /fixes/{session_id}/preview for non-destructive preview generation — expose different transactional semantics: the commit endpoint modifies the session dataframe in place and writes to the audit trail, while the preview endpoint generates a complete preview without touching the session state.

Apache Kafka is deployed using the Confluent Platform distribution, which provides enterprise-grade tooling for topic management, consumer group coordination, and schema registry integration. Zookeeper manages broker coordination and consumer group offset tracking. The kafka-python client library provides the producer and consumer interfaces used within the FastAPI application. Data arrives in configurable batches — defaulting to 500 records per batch — via either a Kafka consumer subscribing to the configured input topic or a direct file upload endpoint that bypasses Kafka for development and testing purposes.

The ML layer uses scikit-learn for all model training and inference. The Random Forest implementations use n_estimators=50 and max_depth=10, balancing correction accuracy against the inference latency requirement of the streaming context. The XAI layer uses the shap library's TreeExplainer for SHAP value computation, which exploits the tree structure of the Random Forest to compute exact Shapley values efficiently without the sampling approximation required for black-box models. The lime library provides the LIME local linear approximation through its LimeTabularExplainer interface.

The React and Vite frontend provides a reactive single-page application for the HITL review workflow. All-channel inactivity detection is implemented through event listeners on pointer, keyboard, mouse, and touch events, resetting a countdown timer on any detected interaction. When the timer expires without detected activity, the frontend triggers a non-destructive preview request to the server and renders the approve/rollback modal presenting the proposed auto-corrections for reviewer confirmation before any modifications are committed.

2.3 Rule Extraction

The LLaMA 3.3-70B model, accessed through the Groq API, forms the foundation of the automated rule extraction module. The Groq inference platform was selected over direct LLM hosting because it provides sub-second inference latency for the 70B parameter LLaMA model — a requirement for integration into a streaming pipeline where rule extraction must complete within the initialization phase before any batches are processed. Self-hosted deployment of a 70B parameter model would require 140 GB of GPU memory in FP16, making it impractical for research prototype environments.

A structured prompt instructs the model to analyze the uploaded domain document and identify field-level validation constraints for each column in the target dataset. The prompt specifies the output format explicitly: a comma-separated CSV with six columns — Column, dtype, min, max, allowed_values, and constraints — allowing direct machine parsing without post-hoc format normalization. The prompt also instructs the model to distinguish between hard constraints (values that are definitively invalid) and soft constraints (values that are statistically unusual but may be legitimate), and to annotate each rule with its evidential basis in the source document.

Post-processing applies three validation steps to the extracted rules before integration into the pipeline. First, dtype consistency is verified: rules specifying min/max bounds for columns annotated as categorical dtype are flagged for expert review. Second, allowed_values lists are normalized — trimmed, lowercased, and deduplicated. Third, constraint syntax is validated against a formal constraint grammar that supports range expressions, enumeration expressions, regex patterns, and logical combinations. Rules that fail validation are quarantined and surfaced in the expert refinement interface rather than silently dropped.

The system supports iterative expert refinement through a dedicated rule management interface in the React frontend. Domain specialists can review automatically extracted rules, add, modify, or delete individual constraints, and promote quarantined rules to active status after manual correction. This hybrid workflow — LLM extraction followed by expert refinement — captures the efficiency of automated extraction while preserving the semantic accuracy of expert oversight.

2.4 Streaming Infrastructure

The streaming infrastructure is implemented using the Confluent Platform distributions of Apache Kafka and Zookeeper. The Kafka producer, running in a separate process, reads the source data stream and publishes records in configurable batches to the configured input topic. The default batch size of 500 records was determined through empirical analysis of the latency-accuracy trade-off: smaller batches reduce the latency between defect occurrence and correction but decrease the amount of context available to the dependency analysis module; larger batches improve dependency analysis quality but increase end-to-end latency.

Fault tolerance is implemented through two mechanisms. First, atomic file writes ensure that each consumed batch is durably written to the consumed_batches directory before the consumer commits its offset, preventing data loss if the consumer crashes between batch consumption and processing completion. Second, the Kafka consumer is configured with manual offset commitment — the consumer reads a batch, processes it, and commits the offset only after the batch has been durably stored — ensuring that uncommitted batches are re-delivered to any restarted consumer instance.

The Kafka producer and consumer are decoupled through the topic abstraction to enable backpressure management. If the consumer falls behind the producer — as may happen when a batch contains an unusually high proportion of defects requiring extensive model training — the unconsumed messages accumulate in the topic with configurable retention. The consumer catches up at its own pace without data loss, and the producer is not blocked by consumer processing delays. This decoupling is the architectural property that makes the framework suitable for production deployment alongside other Kafka consumers sharing the same infrastructure.

2.5 Dependency Analysis and Column Classification

The dependency analysis stage constructs a directed dependency graph over the dataset's columns before any model training occurs. The graph construction process begins by identifying and excluding identifier columns, which should not participate in either the dependency computation or the model training. Three heuristics identify identifier columns: the column name contains the substring 'id'; the uniqueness ratio — the count of distinct values divided by the total row count — exceeds 0.95; or the column contains near-integer values with a sequential increment ratio above 0.90. Excluding identifier columns prevents the models from learning spurious associations with row identifiers that have no predictive meaning for the target attributes.

For the remaining non-identifier columns, pairwise statistical associations are computed using three complementary measures selected to cover all combinations of column data types.

2.5.1 Chi-Square Independence Test

The chi-square test of independence is applied to categorical-categorical column pairs. For a pair of categorical columns X and Y with observed contingency table O and expected table E computed under the independence hypothesis, the test statistic is:

$$\chi^2 = \Sigma_i \Sigma_j (O_{ij} - E_{ij})^2 / E_{ij}, \quad p < 0.05$$

A significance threshold of $p < 0.05$ is applied: column pairs for which the null hypothesis of independence is rejected at this level are retained as dependent pairs in the graph. The chi-square test was selected for categorical pairs because it makes no distributional assumptions about the marginal distributions of either column, requiring only that the expected cell counts are sufficiently large (a condition verified before test application). For column pairs where the chi-square assumptions are violated — due to sparse contingency tables with many cells having expected counts below five — Fisher's exact test is applied as a fallback.

2.5.2 Mutual Information Scoring

Mutual information is used for mixed categorical-numerical column pairs, where one column is categorical and the other is numerical. The mutual information between columns X and Y is:

$$MI(X; Y) = \Sigma_x \Sigma_y p(x,y) \log [p(x,y) / (p(x) p(y))]$$

To ensure comparability across column pairs with different marginal entropies, the mutual information is normalized to [0, 1] using the maximum pairwise mutual information across all column pairs in the dataset:

$$MI\_norm = MI(X; Y) / max_{(i,j)} MI(X_i; X_j) \in [0, 1]$$

A normalized mutual information score above 0.10 is used as the retention threshold for mixed pairs. This threshold was calibrated empirically to balance sensitivity — retaining pairs with genuine predictive relationships — against specificity — excluding pairs where the observed mutual information is attributable to sampling variance in small datasets. For purely numerical-numerical pairs where both columns have continuous distributions, mutual information is estimated using the k-nearest-neighbor entropy estimator.

2.5.3 Spearman Rank Correlation

Spearman's rank correlation is applied to ordinal and continuous numerical column pairs. Unlike Pearson correlation, Spearman's coefficient makes no assumption of linearity or normality, requiring only that the relationship between the two variables is monotonic. The coefficient is computed from the ranks of the observed values:

$$r_s = 1 - (6 \Sigma d_i^2) / (n(n^2 - 1)), \quad |r_s| > 0.30$$

where d_i is the rank difference between the i-th paired observations of the two columns and n is the number of observations. A threshold of $|r_s| > 0.30$ is applied, consistent with conventional guidelines for

characterizing moderate correlation in social and behavioral sciences data. The directional sign of the correlation is retained for documentation purposes but does not affect the dependency graph construction: both positive and negative correlations indicate predictive relationships for the purposes of model feature selection.

Only column pairs that surpass the respective significance threshold for their applicable statistical measure are retained as edges in the dependency graph. The graph is directed: for each target column, the retained predictor columns form its predictor set, and a dedicated Random Forest model is trained using only these validated predictors as input features. This explicit dependency structure is the architectural innovation that enables per-column ML models to produce contextually consistent corrections without contamination from unrelated attributes.

2.6 ML-Based Prediction Engine (Primary Novelty)

The dependency-informed ML prediction engine constitutes the primary technical contribution of this framework. For each non-identifier column identified as an issue target in the current batch, a dedicated supervised learning model is trained using only the column's dependency-validated predictors as input features. The model training occurs within the batch processing window — between batch receipt and HITL review presentation — and must complete within the available latency budget. The lightweight Random Forest configuration (n_estimators=50, max_depth=10) was selected to satisfy this constraint while maintaining correction accuracy.

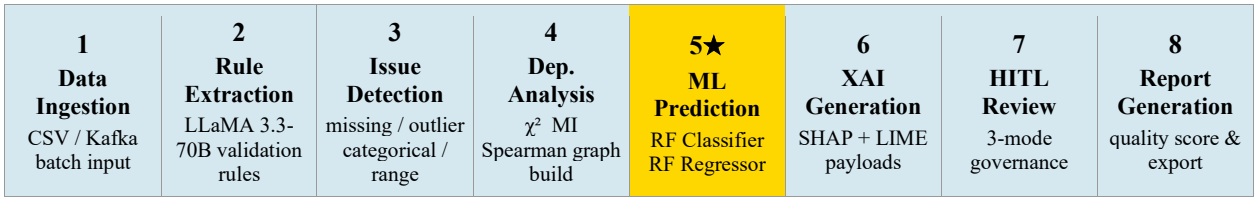

Figure 2-0-billustrates the end-to-end data cleaning pipeline across eight sequential stages, with Stage 5 (ML Prediction) identified as the primary novelty.

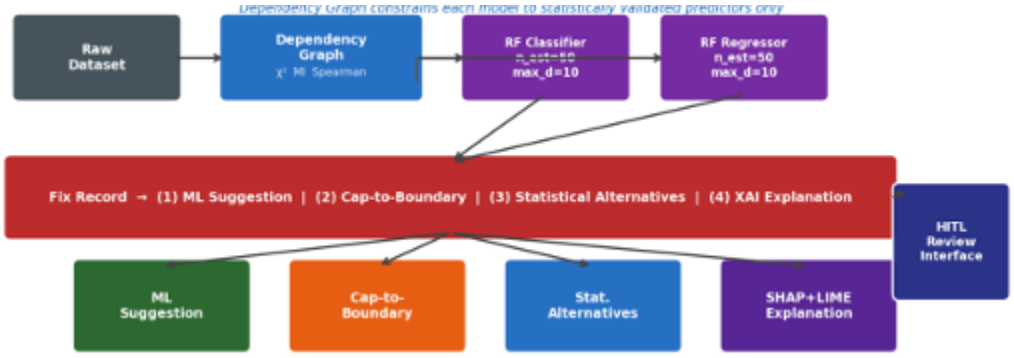

Figure 3-0-illustrates the ML prediction pipeline in detail, showing how the dependency graph informs per-column model training.

Two model variants are instantiated according to the target column's inferred data type. For categorical dtype columns, a Random Forest Classifier is trained on the label-encoded target using the dependency-selected predictor set: RandomForestClassifier(n_estimators=50, max_depth=10, random_state=42). For numeric dtype columns, a Random Forest Regressor is trained after coercing values to float and dropping rows with NaN targets: RandomForestRegressor(n_estimators=50, max_depth=10, random_state=42). The random_state parameter ensures reproducibility of model training across pipeline runs on identical data.

Preprocessing is managed by a scikit-learn Pipeline with a ColumnTransformer that applies type-appropriate transformations to each feature. Numerical predictors pass through a SimpleImputer with strategy=median, which handles missing values in the training set without removing rows. Categorical predictors pass through a SimpleImputer with strategy=most_frequent followed by a OneHotEncoder with handle_unknown=ignore, which produces indicator variables for each observed category while gracefully handling categories in the test batch that were not observed in the training data.

Models are trained only when the clean training set — rows for which the target column is not NaN and all predictor columns have valid values after imputation — contains at least ten rows. This minimum training size threshold prevents models from being trained on insufficient data that would produce unreliable corrections. Columns that fall below this threshold receive no ML prediction; their fix records contain only the statistical alternatives and rule-based suggestions.

For each detected issue, the prediction engine generates a structured fix record comprising four components: the ML suggestion (the primary model prediction), a cap-to-boundary suggestion (the nearest valid boundary value for outlier cases, applicable when the issue is a range violation), a ranked list of statistical alternatives (mean, median, mode, and IQR quartiles computed from the clean training rows), and an explainability payload generated using SHAP and LIME. This four-component fix record serves as the primary input to the HITL review interface, where the reviewer selects among the available options.

The dependency-informed architecture provides a formal basis for why per-column models with restricted predictor sets are expected to outperform global models with unrestricted feature access. Incorporating unrelated columns as predictors introduces spurious correlations that the Random Forest may exploit during training, producing corrections that maximize training-set performance but fail to generalize to the test distribution. This phenomenon is particularly acute for wide datasets where the number of columns approaches or exceeds the number of clean training rows: the model faces a high-dimensional feature space where random correlations are abundant, and the variance of the learned decision boundaries is high. The dependency graph provides principled dimensionality reduction that preserves genuine predictive structure while discarding irrelevant signals.

2.7 XAI Explanation Generation

Every fix suggestion is accompanied by an explainability payload produced through two complementary interpretability methods that address different aspects of the correction reasoning.

SHAP (SHapley Additive exPlanations) computes global feature attributions by assigning each predictor a contribution equal to its average marginal effect on the model output across all possible orderings of the predictors. For tree-based models, the TreeExplainer variant computes exact Shapley values efficiently by exploiting the tree structure to enumerate the contribution of each feature across all paths from root to leaf. The resulting SHAP values quantify both the magnitude and direction of each predictor's contribution to the proposed correction: a positive SHAP value for the 'department' predictor in a salary correction model indicates that the department value of the current record pushed the predicted salary upward relative to the baseline prediction.

The SHAP value formula assigns each feature i a value φ_i defined as its average marginal contribution across all subsets S of the remaining features: $\varphi_i = \Sigma_{S \subseteq F\setminus\{i\}} [|S|!(|F|-|S|-1)!/|F|!] [f(S \cup \{i\}) - f(S)]$, where F is the full feature set and f(S) is the model output using only the features in set S. This formulation satisfies four desirable axioms — efficiency, symmetry, dummy, and linearity — that together make SHAP values the unique fair attribution scheme for cooperative game theory.

LIME (Local Interpretable Model-Agnostic Explanations) generates locally faithful linear approximations of the model's decision surface in the neighbourhood of each specific issue instance. LIME perturbs the input record by sampling new instances near it, obtains the Random Forest's predictions for each perturbed instance, and fits a linear model to the resulting (perturbed instance, prediction) pairs weighted by their proximity to the original record. The linear model's coefficients serve as the local feature attributions, explaining which predictors were most influential for this specific correction decision.

The LIME optimization solves: $\xi(x) = \arg\min_{g \in G} L(f, g, \pi_x) + \Omega(g)$, where f is the Random Forest model, G is the family of linear models, L is a locality-weighted loss measuring how well g approximates f in the neighbourhood defined by π_x, and $\Omega(g)$ is a complexity penalty that encourages sparse explanations with few non-zero coefficients.

Both explanation types are rendered in the HITL review interface alongside the fix suggestions. SHAP covers global feature importance and is most reliable for the average review decision; LIME local approximations address cases where global SHAP attributions may be misleading due to strong feature interactions that make the global average attribution unrepresentative of the model's behaviour in the specific region of input space occupied by the current issue instance. Together they provide robust explanation coverage across diverse model behaviours and input distributions.

2.8 HITL Correction Governance

The HITL governance layer organises correction decisions across three operational modes, selected based on the extent and timing of reviewer engagement. This three-mode structure was designed to accommodate the full range of reviewer availability conditions in production streaming environments: from teams with sufficient staffing for comprehensive record-by-record review to situations where reviewers are temporarily unavailable and automated corrections must proceed to avoid pipeline backpressure.

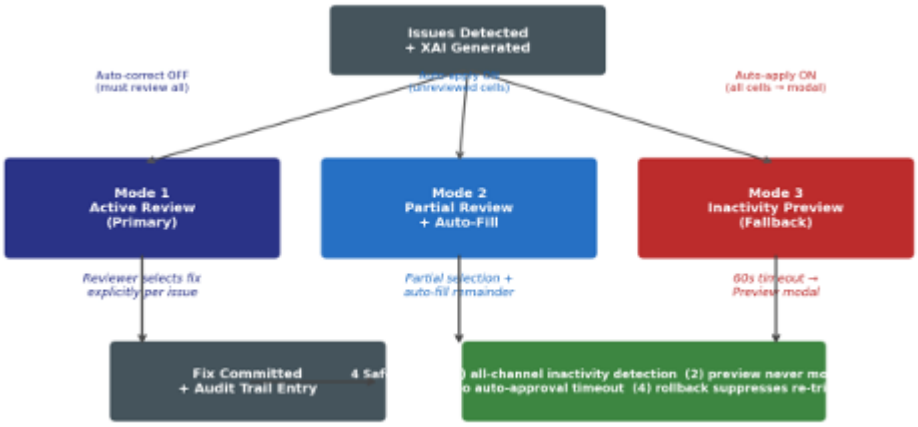

Figure 4-0-dillustrates the three-mode governance decision flow.

Mode 1 — Active Review (primary pathway): The reviewer examines each flagged issue in sequence, inspects the ML suggestion alongside its SHAP and LIME explanation, and explicitly selects one of the available correction options: the ML suggestion, the cap-to-boundary suggestion, one of the statistical alternatives, or a custom value entered manually. No modification is applied to the session dataframe until

the reviewer explicitly submits the full review session. The review interface tracks which issues have been reviewed and which remain pending, displaying a progress indicator to guide efficient review of large batches.

Mode 2 — Partial Review with Auto-Fill (hybrid pathway): The reviewer completes a subset of the available reviews — possibly due to time constraints or a focus on high-priority issues — and submits the session with unreviewed issues remaining. The reviewer-selected corrections are applied exactly as chosen. Unreviewed issues receive auto-selected corrections following the priority ordering: ML suggestion if available, cap-to-boundary for outliers, mode for categorical, median for numerical. All auto-filled corrections are clearly marked in the session audit trail with the auto-fill indicator, distinguishing them from reviewer-approved corrections.

Mode 3 — Inactivity-Triggered Preview (fallback pathway): After the configured inactivity duration — defaulting to 60 seconds — the system detects that no reviewer interaction has occurred and generates a non-destructive server-side preview of the corrections that would be applied using the auto-fill priority ordering. The preview is presented to the reviewer in an approve/rollback modal with a complete list of proposed corrections and their XAI explanations. The reviewer can approve the preview (converting it to committed corrections) or rollback (returning to the active review interface without any modifications).

Four safeguards ensure operational safety in Mode 3. First, all-channel inactivity detection monitors pointer, keyboard, mouse, and touch events simultaneously — a reviewer who moves the mouse without clicking, or who is typing in a different application, will not inadvertently trigger the preview. Second, the preview endpoint never modifies the session dataframe: all proposed corrections exist only in the preview response until the reviewer explicitly approves them. Third, there is no auto-approval timeout: the reviewer's explicit approval action is always required before any preview corrections are committed. Fourth, rollback suppresses re-triggering of the inactivity timer for a configurable cooldown period, preventing the reviewer from immediately receiving another preview after dismissing one.

2.9 Data Quality Scoring

A composite scoring mechanism quantifies dataset reliability at two pipeline checkpoints: before correction, computed from the detected issues in the incoming batch, and after correction, computed from re-detection applied to the corrected dataframe. Computing the score at both checkpoints allows the pipeline to report the quality improvement attributable to the current batch's corrections as a delta, providing a continuous measure of the cleaning pipeline's effectiveness over time.

The score is a weighted sum of three quality dimensions: Completeness (40 percent), Validity (40 percent), and Consistency (20 percent). Table 2 defines the basis of computation for each dimension.

Table 2Data Quality Scoring Dimensions and Weights

Dimension	Weight	Basis of Computation
Completeness	40%	Missing value count relative to total cells (rows × columns)
Validity	40%	Outlier and invalid entry count relative to total cells
Consistency	20%	Duplicate row count and low-variance column detection

The weight assignment reflects the relative severity of each quality dimension for ML pipeline consumers. Missing values and invalid entries — captured by Completeness and Validity — directly corrupt model training data and are assigned equal high weights. Consistency issues — duplicates and low-variance columns — are less immediately harmful and are weighted lower, though they can cause overfitting and reduce model generalization.

The weighted raw quality score is computed as: $Q_{raw} = 0.40 \cdot S_C + 0.40 \cdot S_V + 0.20 \cdot S_K$, where S_C, S_V, and S_K denote the Completeness, Validity, and Consistency sub-scores respectively. Each sub-score is computed as 1 minus the defect rate for its dimension, scaled to [0, 100]. The raw score is then normalized to the [0, 95] range using: $Q = \min(95, [(Q_{raw} - Q_{min}) / (Q_{max} - Q_{min})] \times 95)$. The upper bound of 95 rather than 100 reflects the irreducible noise inherent in high-velocity streaming data: in production streaming environments, achieving a perfect 100 score is infeasible, and a maximum of 95 calibrates reviewer expectations appropriately.

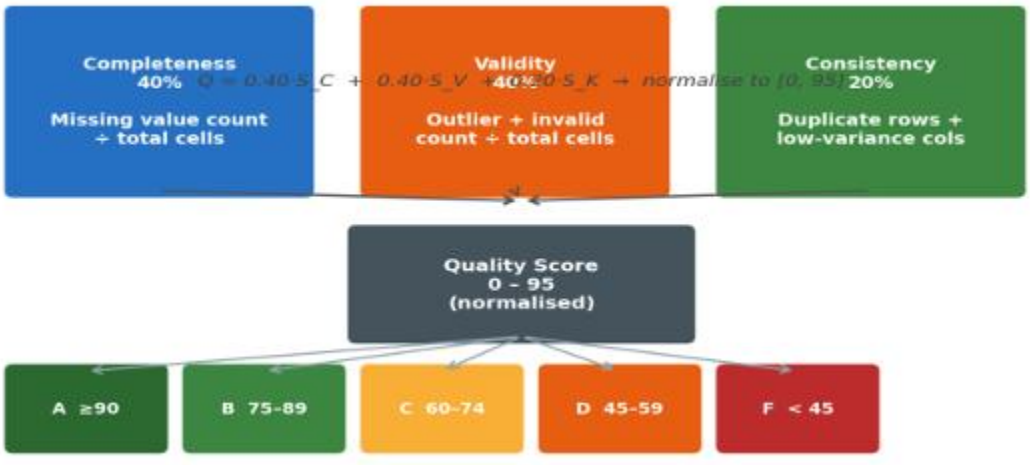

Figure 5-eillustrates the composite scoring model, showing the three weighted dimensions, the combination formula, and the five-tier grade mapping.

The normalized score is mapped to a five-tier grade scale: A (90 and above), B (75 to 89), C (60 to 74), D (45 to 59), F (below 45). The grade thresholds were calibrated against the quality characteristics of representative streaming datasets across three domains — healthcare records, financial transactions, and IoT sensor readings — to ensure that Grade A corresponds to datasets suitable for direct use in model training without additional cleaning, and Grade F corresponds to datasets requiring human review of the entire dataset rather than automated batch correction.

03. RESULTS AND DISCUSSION

3.1 Experimental Evaluation Framework

The framework is evaluated across six complementary dimensions that collectively assess all five research questions. The evaluation design separates concerns that might confound each other in a single end-to-end measurement: for example, correction accuracy is measured against synthetic ground truth rather than against reviewer judgement, ensuring that the ML prediction quality is assessed independently of reviewer behaviour.

Table 3 summarizes the six evaluation dimensions, their measurement methodologies, and the datasets used for each.

Table 3Evaluation Metrics Across Six Dimensions

Dimension	Metric	Dataset / Method
Rule Extraction Fidelity	Precision, Recall, F1 vs. ground-truth annotations	Healthcare, Financial, IoT domain corpora (3 datasets)
Issue Detection	Sensitivity, Specificity, F1 per anomaly category	Synthetically corrupted benchmarks with known ground truth
Correction Accuracy	MAE (numerical), Classification Accuracy (categorical)	Synthetic corruption of held-out clean dataset rows
Dependency Precision	Edge validation rate vs. domain expert annotations	Expert-annotated dependency graphs for 3 datasets
HITL Review Efficiency	Issues resolved via human vs. auto-fallback; time-to-review	User study with domain expert reviewers
Throughput and Latency	Records per second; per-batch latency (ms)	Batch sizes 100 / 500 / 1000; widths 10 / 50 / 100 cols

3.1.1 Rule Extraction Fidelity

Rule extraction fidelity is measured by precision and recall of extracted validation constraints against ground-truth domain rule annotations prepared by subject matter experts for three domain corpora: a clinical data dictionary from a hospital electronic health record system, a financial data governance document from a banking compliance department, and a sensor configuration specification from an industrial IoT deployment.

Precision is defined as the fraction of extracted rules that correspond to a genuine constraint in the ground-truth annotation set. A rule is considered to match a ground-truth constraint if it specifies the correct column,

the correct constraint type (range, enumeration, or regex), and a constraint boundary that differs from the ground-truth by no more than the specified tolerance (5 percent for numerical boundaries, exact match for categorical values). Recall is the fraction of ground-truth constraints for which the extraction module produces at least one matching rule. F1 is the harmonic mean of precision and recall.

Rule extraction precision is expected to be highest for well-documented domains such as healthcare and financial, where regulatory requirements mandate explicit constraint documentation in standardized formats. For IoT domains with idiosyncratic sensor-specific schemas and sparse documentation, lower precision is anticipated due to the model's reliance on implicit contextual cues absent from terse technical specifications.

3.1.2 Issue Detection Performance

Issue detection performance is measured on synthetically corrupted benchmark datasets with known ground truth. Clean versions of three publicly available structured datasets are selected, and defects are injected programmatically at a controlled defect rate of 5 to 15 percent per column. Four defect categories are injected: missing values (null replacement), statistical outliers (IQR-based injection of values 3 to 5 standard deviations from the mean), categorical violations (replacement with values outside the allowed enumeration), and range violations (replacement with values outside the defined min/max boundaries).

Sensitivity (true positive rate), specificity (true negative rate), and F1-score are computed separately for each defect category and aggregated across all three benchmark datasets. The per-category analysis is essential because the detection mechanisms differ: missing values are detected through null checks, outliers through IQR-based statistical thresholds, and categorical violations through rule constraint matching.

3.1.3 Correction Accuracy

Correction accuracy is measured by comparing model-imputed values against the original uncorrupted values for the synthetically corrupted cells. For numerical attributes, Mean Absolute Error (MAE) is computed as the mean absolute difference between the imputed value and the original value, normalized by the standard deviation of the column in the clean dataset. For categorical attributes, classification accuracy is the fraction of corrected cells where the imputed category matches the original category.

The dependency-informed per-column models are compared against three baselines: mean/mode imputation (standard statistical practice), a global Random Forest trained on all non-target columns (isolating the effect of dependency filtering), and HoloClean (where applicable for the batch-mode baseline). The comparison against the Global RF baseline is the most informative for evaluating the core technical claim of the framework: it holds the model type constant while varying only the feature selection strategy.

3.2 Baseline Comparisons

The framework is evaluated against four baselines of increasing sophistication. Table 3 summarizes the baseline configurations and their evaluation purposes.

Table 4Baseline Comparison Methods

Baseline	Configuration	Purpose
Rule-Only	Extracted rules, no ML prediction	Lower-bound: detection without repair
Statistical Imputation	Mean/mode imputation, no dependency analysis	Standard preprocessing practice benchmark
Global RF	Random Forest on all non-target columns	Isolates dependency selection quality gain
Full Auto (no HITL)	Full ML pipeline, no human review	Quantifies HITL governance value

The Rule-Only baseline establishes the lower bound for correction quality achievable through constraint enforcement alone, without any ML-based imputation. It applies only those corrections mandated by extracted rules — replacing values outside defined ranges with the nearest boundary, replacing categorical violations with the most frequent valid category — and leaves missing values unfilled. This baseline isolates the contribution of ML prediction to correction quality beyond what rule enforcement alone achieves.

The Statistical Imputation baseline represents standard preprocessing practice in production ML pipelines. Missing values are filled with the column mean for numerical attributes and the column mode for categorical attributes; outliers are clipped to the IQR boundaries. No dependency information is used, and all corrections are applied automatically without human review. This baseline reflects the status quo that the proposed framework seeks to improve upon.

The Global RF baseline is the most informative comparator for evaluating the dependency-informed architecture. It trains a Random Forest model for each target column using all non-target columns as predictors — the same model type and hyperparameters as the proposed framework, but without dependency filtering. By holding the model type constant and varying only the feature selection strategy, this comparison isolates the quality contribution of the dependency graph as a distinct variable. If the dependency-informed models outperform the Global RF models on the correction accuracy metrics, this provides direct evidence for the central technical claim.

The Full Auto baseline applies the complete ML prediction pipeline without any human review. All corrections — ML suggestion, cap-to-boundary, or statistical alternative, selected by the priority ordering — are committed automatically without presenting the fix record to a human reviewer. Comparing the Full

Auto baseline against the proposed framework's HITL-enabled corrections quantifies the governance value of human review: the quality delta between human-selected and auto-selected corrections for each anomaly category.

3.3 Expected Results and Evaluation Design Considerations

Based on the theoretical properties of dependency-informed feature selection and the design of the HITL governance layer, the following expected results inform the evaluation design. These expectations are grounded in the statistical properties of the dependency measures and in published results from related systems.

Table 5Expected Performance Summary

Metric	Expected Outcome	Basis of Expectation
Correction Accuracy vs Global RF	Dep.-informed superior for $\rho > 0.3$ col pairs	Dimensionality reduction theory; bias-variance trade-off
Rule Extraction Fidelity (Healthcare)	Precision > 0.85, Recall > 0.75	LLaMA-70B benchmark on structured domain text
Rule Extraction Fidelity (IoT)	Precision ~0.65, Recall ~0.60	Sparse documentation; idiosyncratic schemas
HITL vs Auto-Fill Quality Delta	Human > Auto for categorical and outlier issues	Domain knowledge unavailable to statistical models
Throughput (500-record batch, 50 cols)	< 2 s per-batch latency	RF inference: ~10 ms/model; 50 cols = ~500 ms total
Throughput scaling (2× consumers)	~2× throughput	Kafka consumer group linear partition scaling

The correction accuracy advantage of dependency-informed models is expected to be most pronounced for datasets with moderate to high inter-column correlation, where the Global RF baseline's unrestricted feature access leads to pronounced overfitting to spurious correlations. For datasets where columns are genuinely independent — such as randomly generated synthetic data — the dependency graph will be sparse, most columns will lack a trained corrective model, and both the proposed framework and the Global RF baseline will fall back to statistical alternatives. In this pathological case, the two approaches are expected to perform equivalently, confirming that the dependency-informed architecture provides graceful degradation rather than degraded performance when no genuine dependencies are present.

The HITL quality advantage is expected to be most pronounced for outlier and categorical violation corrections. Outlier corrections require semantic judgement about whether an extreme value is a genuine measurement or a data error — a distinction that statistical models cannot make without domain context. A

salary of $500,000 in a dataset of retail employee records is almost certainly an error; the same value in a dataset of executive compensation records may be valid. Categorical violation corrections require knowledge of which replacement category is semantically appropriate in context — replacing a missing job title with 'Unknown' is appropriate if 'Unknown' is a valid category, but replacing it with the most frequent category may assign an incorrect title that corrupts downstream analyses.

3.4 Discussion

3.4.1 The Dependency-Informed Architecture

The central technical claim of this framework is that correcting a column using only its statistically validated predictors produces superior corrections compared to using all available columns indiscriminately. This claim rests on two well-established mechanisms from statistical learning theory.

The first mechanism is the curse of dimensionality as applied to classification and regression. When the number of input features is large relative to the number of training samples, the volume of the feature space becomes so vast that the available training points are sparse, and the variance of any learned decision boundary is high. Even powerful ensemble methods like Random Forest are affected: the individual trees in the forest are trained on random subsets of features, but if all features are included in the full feature set, the random subsets may still contain many irrelevant features that introduce noise. The dependency graph's feature restriction reduces the effective dimensionality of the feature space to only those dimensions that genuinely co-vary with the target, reducing the variance of the learned boundary without sacrificing the predictive structure that enables accurate correction.

The second mechanism is cross-validation generalization. A model trained on a restricted, relevant feature set generalizes more reliably to unseen data than a model trained on an unrestricted feature set that includes spurious correlations. Spurious correlations — statistical coincidences that exist in the training sample but do not reflect genuine causal or associative relationships — inflate training-set performance metrics while degrading test-set performance. The dependency graph explicitly excludes feature pairs whose statistical association does not exceed the significance thresholds, providing a principled safeguard against spurious correlation exploitation.

For datasets that are genuinely independent column-wise — where the dependency graph is sparse — the framework degrades gracefully by falling back to statistical alternatives and rule-based corrections. This property is important for production robustness: the framework does not claim to improve all datasets, only those where genuine inter-column dependencies exist. The dependency graph itself serves as a diagnostic tool: a sparse graph indicates that the dataset is near-independent column-wise, providing actionable information about its structure even when ML-based correction cannot be applied.

3.4.2 Governance and Accountability

The three-mode HITL governance structure generates a traceable record that distinguishes which corrections were reviewer-approved, which were auto-filled for unreviewed issues, and which were applied through the inactivity-triggered fallback. This distinction carries concrete consequences in regulated domains that extend beyond technical data quality.

In healthcare settings, a correction to a patient record approved by a named clinical reviewer satisfies data lineage requirements more completely than an auto-applied correction, even if both yield the same numerical outcome. Regulatory frameworks such as HIPAA require that modifications to protected health information be traceable to an identifiable responsible party. The audit trail generated by the proposed framework captures the governance mode, the reviewer's session identifier, the timestamp, and the specific correction selected for every record in every batch, providing the evidentiary basis for such traceability.

In financial settings, corrections to transaction records may constitute retroactive modification of financial data, which is subject to audit requirements under regulations such as SOX and Basel III. The audit trail's distinction between Mode 1 (reviewer-approved), Mode 2 (partial review with documented auto-fill), and Mode 3 (inactivity-triggered preview with explicit approval) provides the granularity of accountability required by financial data governance frameworks. The non-destructive preview architecture ensures that even Mode 3 corrections require explicit reviewer confirmation, preventing any automated modification without an identifiable human approval action.

3.4.3 XAI as a Review Enabler

The combination of SHAP and LIME explanations in the HITL review interface serves a functional rather than cosmetic role. Without explanations, a reviewer facing a suggested correction — 'impute missing salary with $72,500' — must accept or reject the suggestion based entirely on their prior domain knowledge and intuition. With SHAP explanations, the reviewer can see that the model's prediction was driven primarily by the 'department' attribute (contribution: +$15,000) and the 'seniority_level' attribute (contribution: +$8,000), providing a transparent reasoning chain that allows the reviewer to validate or challenge the suggestion in seconds.

The complementary roles of SHAP and LIME address different failure modes of model explanation. SHAP covers global feature attributions that represent the average contribution of each feature across the full training distribution; these attributions are reliable for instances that are representative of the training distribution. LIME local approximations address cases where global SHAP attributions may be misleading due to strong feature interactions: in regions of the input space where the relationship between features and predictions is highly nonlinear, the linear LIME approximation provides a locally faithful explanation that

captures the model's behaviour in the immediate neighbourhood of the current instance, even if it does not accurately represent the global model.

The combination of both methods in the review interface allows reviewers to develop calibrated trust in the model's suggestions — accepting corrections where both SHAP and LIME attributions align with domain knowledge, and scrutinising or overriding corrections where the attributions reveal surprising or counterintuitive feature dependencies. This calibrated trust is more valuable than either blind acceptance or blanket scepticism of automated corrections.

3.4.4 Scalability Considerations

Dependency computation scales quadratically with the number of non-identifier columns: for a dataset with n non-identifier columns, n(n-1)/2 pairwise statistical tests must be computed. For wide datasets with 100 columns, this requires approximately 4,950 statistical tests, which constitutes the dominant initialization overhead of the framework. The tests themselves are computationally lightweight — chi-square and Spearman tests on column vectors are O(n) operations — but the quadratic growth in the number of tests creates a practical constraint for very wide datasets.

Production deployments in wide-schema environments may benefit from pre-specified dependency graphs derived from domain knowledge, which bypass the statistical computation entirely and provide the model training stage with a curated predictor set based on established subject-matter relationships. Alternatively, dimensionality reduction prior to dependency analysis — projecting the column set onto principal components or applying hierarchical clustering to identify column groups — can reduce the effective number of columns for dependency computation while preserving the most predictively important structure.

The Kafka-based decoupling between data producers and the processing backend provides architectural headroom for throughput scaling. Adding additional consumer instances — which join the existing consumer group and automatically receive a proportional share of the topic's partitions — increases throughput linearly without requiring any modification to the producer, the processing logic, or the storage layer. This horizontal scaling property is a direct consequence of the Kafka consumer group architecture and requires no changes to the framework implementation.

3.4.5 Comparison with HoloClean

Relative to HoloClean, the proposed framework offers three structural advantages that make it more suitable for production streaming deployment. First, streaming-native execution: HoloClean is designed for batch-mode operation and requires per-batch model re-initialization, which is incompatible with the latency constraints of a streaming pipeline. The proposed framework's per-column Random Forest models are

trained incrementally as new batches arrive, with model checkpointing enabling warm-starting of subsequent batch models from prior state.

Second, automated rule derivation: HoloClean presupposes a set of manually specified integrity constraints that define the space of valid corrections. For novel or rapidly evolving data domains where constraint documentation does not exist or is out of date, this requirement is a blocking obstacle. The proposed framework's LLM-based rule extraction module derives a constraint vocabulary from available documentation without manual specification, enabling deployment in domains where HoloClean cannot operate.

Third, per-correction human accountability: HoloClean applies corrections automatically without any mechanism for human review or approval at the individual correction level. The proposed framework's HITL governance layer ensures that every correction decision is traceable to either a human reviewer's explicit selection or an audited automated pathway, providing the accountability that regulated domains require.

The one limitation of the proposed framework relative to HoloClean is that HoloClean's joint probabilistic repair enforces multi-constraint consistency in a single optimization pass, ensuring that the corrected dataset simultaneously satisfies all specified constraints. The proposed framework applies corrections sequentially — column by column, defect by defect — and does not guarantee global joint consistency across all constraints simultaneously. A correction to one column may violate a constraint that involves another column whose correction has not yet been applied. This limitation motivates the future work on joint consistency enforcement described in the conclusions.

04. CONCLUSION

4.1 Conclusions

This dissertation has presented a scalable, intelligent, and human-governed framework for structured data cleaning in streaming environments. The framework's primary technical novelty is a dependency-informed ML prediction engine in which each target column receives a dedicated Random Forest model trained exclusively on statistically validated predictors, identified through chi-square independence testing for categorical pairs, normalized mutual information scoring for mixed pairs, and Spearman rank correlation for numerical pairs. This architecture prevents the spurious corrections that arise when unrelated columns contaminate the feature space, and it provides transparent feature attribution through SHAP and LIME explanations that make correction reasoning auditable by human reviewers.

Three mutually reinforcing contributions combine to produce a system whose quality improvement exceeds what any single component achieves in isolation. LLM-based rule extraction via LLaMA 3.3-70B supplies the constraint vocabulary governing detection and auto-correction validation, removing the manual specification burden that makes rule-based systems impractical for novel or rapidly evolving data domains. Dependency-informed ML prediction generates accurate, contextually consistent correction candidates by restricting each model's feature access to the columns with genuine predictive relationships, eliminating the dimensionality and spurious correlation problems that degrade holistic ML-based repair frameworks on wide datasets. HITL governance ensures corrections are both contextually appropriate and accountable, distinguishing reviewer-approved corrections from automated fallback corrections in a per-correction audit trail that satisfies the data lineage requirements of regulated domains.

The composite quality scoring mechanism, with Completeness and Validity each weighted at 40 percent and Consistency at 20 percent, provides an honest quantification of dataset reliability calibrated to the irreducible noise inherent in high-velocity streaming data. The normalization to 0–95 and the five-tier grade mapping (A through F) give pipeline operators an actionable quality signal that distinguishes datasets ready for direct model training from those requiring additional intervention.

The production-grade Kafka streaming backbone, implemented using the Confluent Platform with configurable batch sizes and fault tolerance through atomic file writes, provides the infrastructure for deploying the framework at realistic streaming velocities. The decoupling between the data ingestion layer and the processing layer through the Kafka topic abstraction enables horizontal throughput scaling by adding consumer instances without modifying any component of the processing or storage layers.

The deliberate asymmetry between the LLM-based rule extraction — which operates on unstructured text and requires semantic understanding — and the rule-based dependency analysis — which operates on numerical column statistics and requires no linguistic intelligence — reflects a broader principle for the design of hybrid AI systems. The appropriate mechanism for each task is the one matched to the nature of the information being processed. LLMs provide value for tasks that require reading, interpreting, and generalizing from text; statistical methods provide value for tasks that can be reduced to well-defined numerical computations. The proposed framework applies each mechanism precisely where it provides genuine advantage and avoids applying LLMs to tasks that are better handled by deterministic computation.

The evaluation framework, designed across six dimensions spanning rule extraction fidelity, issue detection performance, correction accuracy, dependency graph precision, HITL review efficiency, and system throughput, provides a comprehensive assessment methodology for validating all five research questions. The Global RF baseline comparison is particularly important for isolating the quality contribution of dependency-informed feature selection as a distinct variable, holding the model type constant to avoid confounding the evaluation with model selection effects.

4.2 Recommendations and Future Work

The following directions for future investigation are identified based on the design decisions, evaluation framework, and limitations of the current implementation.

1. Empirical Evaluation with Human Subjects: The evaluation framework described in this dissertation includes a planned user study measuring time-to-review per issue and quality delta of human versus auto-corrections. This study should be conducted with domain expert reviewers from the target application domains — clinical informaticists for healthcare datasets, compliance analysts for financial datasets, and systems engineers for IoT datasets — to ensure that the measured quality delta reflects genuine domain knowledge rather than generic data cleaning intuition.

2. Online Learning Integration: The current framework treats each batch independently — models are trained on the clean rows of the current batch and do not carry information across batches. Integrating online learning would allow reviewer-approved corrections to be accumulated as labelled training examples and incorporated into updated model checkpoints, enabling the framework to improve its correction accuracy continuously as more batches are processed. A Kafka-backed correction stream could feed reviewer selections to an online Random Forest update procedure without disrupting the real-time processing of incoming batches.

3. Joint Consistency Enforcement: The most significant technical limitation of the current framework relative to HoloClean is the absence of joint consistency enforcement across all constraints simultaneously.

Future work should extend the correction stage with a constraint satisfaction layer that takes the set of per-column corrections proposed by the ML prediction engine and solves for the globally consistent assignment that satisfies all active constraints. Integer linear programming and constraint propagation are candidate approaches for this extension, with the trade-off between solution quality and computation time determining the applicable batch size and throughput.

4. Retrieval-Augmented Rule Extraction: The current rule extraction module prompts LLaMA 3.3-70B directly with the full domain document, which may exceed the model's effective context window for very long specifications. A RAG architecture that chunks the document into semantically coherent segments, embeds them in a vector database, and retrieves only the most relevant segments for each column's constraint extraction would improve extraction fidelity for long documents while reducing the token budget consumed per extraction call.

5. Schema Drift Integration: Streaming environments are subject to schema drift — the gradual evolution of the upstream data schema as source systems add new fields, modify value ranges, and retire old columns. The current framework operates under the assumption of a stable schema across the evaluation period. Future work should integrate structural drift detection that monitors the distribution of column names, data types, and value ranges across consecutive batches and alerts operators when schema drift is detected, triggering re-extraction of rules and reconstruction of the dependency graph.

6. Differential Privacy for Regulated Domains: In GDPR- and HIPAA-governed deployments, the auto-correction fallback modes (Mode 2 auto-fill and Mode 3 inactivity-triggered preview) apply corrections that are visible to the pipeline operators. For datasets containing protected health information or personally identifiable information, even the correction process may require privacy protection. Future work should incorporate differential privacy mechanisms into the auto-correction fallback, adding calibrated noise to numerical corrections to prevent inference about individual record values from the correction decisions.

7. Adaptive Inactivity Threshold: The current inactivity timeout is set as a fixed configuration parameter (default 60 seconds). An adaptive threshold that learns from the distribution of historical review times — setting the timeout at, say, the 90th percentile of observed inter-action intervals — would reduce false triggers of the inactivity preview for reviewers who are actively engaged but processing complex corrections slowly. Bayesian estimation of the inter-action interval distribution from the reviewer's session history could provide a principled basis for the adaptive threshold.

REFERENCES

[1] W. Fan and F. Geerts, Foundations of Data Quality Management. San Rafael, CA: Morgan & Claypool, 2012.

[2] G. Cong, W. Fan, F. Geerts, X. Jia, and S. Ma, "Improving data quality: Consistency and accuracy," in Proc. 33rd VLDB, Vienna, 2007, pp. 315–326.

[3] O. Troyanskaya et al., "Missing value estimation methods for DNA microarrays," Bioinformatics, vol. 17, no. 6, pp. 520–525, Jun. 2001.

[4] J. Yoon, J. Jordon, and M. van der Schaar, "GAIN: Missing data imputation using generative adversarial nets," in Proc. ICML, Stockholm, 2018, pp. 5689–5698.

[5] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," Proc. VLDB Endow., vol. 10, no. 11, pp. 1190–1201, Aug. 2017.

[6] I. F. Ilyas and X. Chu, Data Cleaning. San Rafael, CA: Morgan & Claypool, 2019.

[7] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," Proc. VLDB Endow., vol. 9, no. 12, pp. 948–959, 2016.

[8] Y. Li et al., "Table-GPT: Table-tuned GPT for diverse table tasks," arXiv preprint arXiv:2310.09263, Oct. 2023.

[9] P. Peeters, R. Balayn, and C. Lofi, "Entity matching using large language models," arXiv preprint arXiv:2310.11244, Oct. 2023.

[10] A. Bontempelli et al., "Human-in-the-loop handling of knowledge drift," Data Min. Knowl. Discov., 2022.

[11] X. Zhu, J. Wu, and S. Chen, "Real-time data quality monitoring for streaming data," in Proc. IEEE Int. Conf. Big Data, Los Angeles, 2019, pp. 2812–2821.

[12] A. Bifet, G. Holmes, and B. Pfahringer, "MOA: Massive online analysis," J. Mach. Learn. Res., vol. 11, pp. 1601–1604, Aug. 2010.